

Setting Up GitHub

Account, SSH Keys, and First Connection

Dario Pescini

Università degli Studi di Milano-Bicocca
Dipartimento di Statistica e Metodi Quantitativi

Section 1

GitHub Account

Create a GitHub Account

- 1 Go to github.com *Sign up*
- 2 Choose a username — pick something professional, you will share it with collaborators and future employers
- 3 Select the **Free** plan
- 4 Verify your email address

GitHub is free for public repositories and for private repositories with up to 3 collaborators.

Why Not Just Use a Password?

GitHub **discontinued password authentication** for Git operations in 2021.

Two alternatives:

Method	When to use
SSH key	Daily work from a fixed machine
Personal Access Token (PAT)	Scripts, CI/CD, temporary access

We will set up **SSH** — it is the most comfortable for interactive use.

Section 2

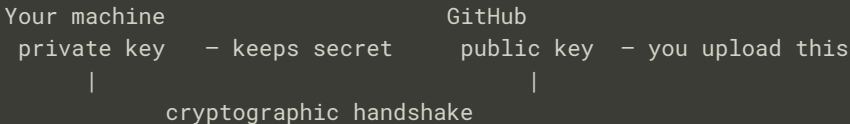
SSH Keys

What Is an SSH Key?

SSH (Secure Shell) authentication uses a **key pair**:

- **Private key** – stays on your machine, never shared
- **Public key** – uploaded to GitHub, can be shared freely

When you connect, GitHub challenges your machine with a message that only the private key can sign. No password is ever transmitted.



Step 1 — Check for Existing Keys

Open a terminal:

```
ls ~/.ssh
```

If you see `id_ed25519` and `id_ed25519.pub` (or `id_rsa / id_rsa.pub`) you already have a key pair — skip to Step 3.

Otherwise, generate a new one.

Step 2 — Generate a Key Pair

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

- Accept the default file location (press Enter)
- Set a passphrase (recommended) or leave it empty

This creates two files in `~/ .ssh/`:

File	Content
<code>id_ed25519</code>	Your private key — never share this
<code>id_ed25519.pub</code>	Your public key — you will upload this

Step 3 — Add the Key to the SSH Agent

The agent holds your key in memory so you do not need to type your passphrase at every push.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Step 4 — Copy Your Public Key

```
cat ~/.ssh/id_ed25519.pub
```

Copy the entire output — it looks like:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAA... your_email@example.com
```

Select all and copy to clipboard (Ctrl+Shift+C in the terminal).

Step 5 — Add the Key to GitHub

- 1 GitHub top-right avatar **Settings**
- 2 Left sidebar **SSH and GPG keys**
- 3 Click **New SSH key**
- 4 Title: something descriptive (e.g. Ubuntu VM Lab)
- 5 Paste your public key **Add SSH key**

Step 6 — Test the Connection

```
ssh -T git@github.com
```

Expected response:

```
Hi username! You've successfully authenticated,  
but GitHub does not provide shell access.
```

If you see this, your SSH setup is complete.

Section 3

Personal Access Tokens

Alternative: Personal Access Token (PAT)

Use a PAT when SSH is not practical (e.g. a shared machine or an automated script).

Generate a PAT:

- 1 GitHub Settings **Developer settings**
- 2 **Personal access tokens** Tokens (classic) Generate new token
- 3 Select scope: at minimum check **repo**
- 4 Copy the token immediately – GitHub will not show it again

Use the token as your **password** when Git asks for credentials over HTTPS.

Storing a PAT Safely

Never paste a token directly into a script or commit it to a repo.

Store it in your system credential manager:

```
git config --global credential.helper store
```

The first time you push, Git will ask for username + token and store it in ~/.git-credentials.

For lab machines: use cache instead of store so the token is only kept in memory for a limited time.

```
git config --global credential.helper 'cache --timeout=3600'
```

Section 4

Summary

Setup Checklist

One-time setup

- Create GitHub account
- Generate SSH key pair
- Add public key to GitHub
- Test connection

Per-machine setup

- `git config --global user.name`
- `git config --global user.email`
- Start ssh-agent (or add to `~/.bashrc`)

Once done, you will never need to type a password for GitHub again.